SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3 – TRANSFORMER MECHANICS WORKSHEET

Course Code:	CSA6506	**Course Title:**	Generative AI and Large Language Models
CO Assessed:	CO1 – Precision (BL1, BL2)	**Assessment:**	Assessment Tool 3 – Transformer Mechanics Worksheet
Weightage:	40% of CIA	**Total Marks:**	100
CO1 Statement:	*Explain the fundamentals of Artificial Intelligence, Generative AI, Foundation Models, Transformer architecture, tokenization, embeddings, and Large Language Models. (BL1, BL2)*		
Student Name:	G. PRANATHI	**Reg. No.:**	192472237
Section / Batch:	CSE AI/2024	**Date:**	

Instructions to Students

• Answer all questions. It carries 100 Marks. Total: 100 Marks.

· Tokenize a sentence.
· Convert tokens to token IDs.
· Explain embeddings.
· Label a Transformer encoder.
· Identify Query, Key, and Value.
· Explain which words receive higher attention.
· Compare Transformer models.
· Complete a Transformer pipeline.
· Draw a block diagram.

Code of Conduct

I G.PRANATHI [192472237] certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

TRANSFORMER MECHANICS WORKSHEET

Question 1: Sentence Tokenization

Consider the sentence: “Transformers enable intelligent text generation.”

Split the sentence into tokens using a suitable tokenizer. Explain how the tokenization may differ between a word-level tokenizer and a subword tokenizer.

Topic: Tokenization | Bloom’s level: Understand

Question 2: Token-to-ID Conversion

Given the token sequence `["AI", "models", "learn"]` and the vocabulary mapping `{"AI": 12, "models": 25, "learn": 8}`, convert the tokens into their corresponding token IDs. What happens if a token is not present in the vocabulary?

Topic: Token IDs | Bloom’s level: Apply

Question 3: Understanding Embeddings

Explain how token IDs are converted into embedding vectors in a Transformer. Why are embedding vectors more useful for language processing than token IDs alone?

Topic: Token embeddings | Bloom’s level: Understand

Question 4: Transformer Encoder Labelling

Draw a Transformer encoder block and label the following components in the correct order: multi-head self-attention, input embeddings with positional information, feed-forward neural network, residual connections, and layer normalization.

Topic: Transformer encoder architecture | Bloom’s level: Apply

Question 5: Query, Key, and Value Identification

In the sentence “The student submitted the assignment because it was complete,” explain how Query, Key, and Value vectors are generated from token representations. Describe their respective roles in determining relationships between words.

Topic: Query–Key–Value | Bloom’s level: Analyze

Question 6: Attention Weight Analysis

Consider the sentence: “The cat chased the mouse because it moved quickly.” Which words might receive higher attention when processing the word “it”? Justify your answer and explain how attention weights help a Transformer capture contextual relationships.

Topic: Self-attention | Bloom’s level: Analyze

Question 7: Comparing Transformer Models

Compare BERT, GPT, and T5 in terms of their attention approach, input-output behaviour, and typical applications. Identify which model would be most suitable for (a) text classification, (b) open-ended text generation, and (c) text summarization, with justification.

Topic: Transformer model comparison | Bloom's level: Analyze

Question 8: Completing the Transformer Pipeline

Complete the following Transformer processing pipeline by filling in the missing stages:

Text Input → _____ → _____ → Embedding and Positional Information → _____ → Output Probabilities → Predicted Token.

Explain the function of each missing stage.

Topic: Transformer pipeline | Bloom's level: Apply

Question 9: Transformer Block Diagram

Draw a block diagram illustrating how an input sentence is processed by a Transformer-based language model to generate an output. Include tokenization, token ID conversion, embeddings, attention layers, feed-forward layers, output projection, and token prediction.

Topic: Transformer architecture | Bloom's level: Create

Question 10: Integrated Transformer Mechanics

A language model receives the sentence "Generative AI creates new content." Trace the sentence through tokenization, token ID conversion, embedding generation, self-attention, contextual representation, and output prediction. Explain how the attention mechanism helps the model understand relationships among the words.

Topic: Integrated Transformer pipeline | Bloom's level: Analyze

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Mechanics Explanation	30%	All key concepts (AI, ML, DL, GenAI, LLMs) correctly identified with clear hierarchical relationships	Most concepts correct; minor gaps in relationships	Several concepts missing or misclassified	Major conceptual errors; map incomplete
Worked-Example Accuracy	25%	Real-world examples linked accurately to each concept	Examples mostly relevant	Few or generic examples	No real-world linkage shown
Mathematical Reasoning	20%	Logical, well-organized structure with clear connectors	Mostly organized; minor structural issues	Some disorganization affecting clarity	Poorly structured, hard to follow
Diagram & Working Clarity	15%	Visually clear, creative, easy to interpret	Neat and readable	Cluttered or inconsistent formatting	Untidy, difficult to interpret
Step Justification	10%	Concise labels/notes that add clarity	Adequate labeling	Minimal or unclear labeling	No supporting labels or explanation

Marks Evaluation:

Mechanics Explanation (30%)	Worked-Example Accuracy(25%)	Mathematical Reasoning (20%)	Diagram & Working Clarity (15%)	Step Justification (10%)	Total Marks (100)

SOLUTION: TRANSFORMER MECHANICS WORKSHEET

QUESTION 1: Tokenization

Tokenization is the process of dividing a sentence into smaller units called tokens. These tokens may be words, parts of words, or punctuation marks.

Example sentence: “AI helps people learn.”

Word-level tokens: [“AI”, “helps”, “people”, “learn”, “.”]

Subword-level tokens: [“AI”, “help”, “##s”, “people”, “learn”, “.”]

The exact tokens depend on the tokenizer used.

Word-level Tokenization	**Subword Tokenization**
Splits text into complete words.	Splits text into words or smaller parts.
Requires a larger vocabulary for rare words.	Can handle many rare and new words.
Unknown words may be difficult to represent.	Can combine known subwords to represent unfamiliar words.

Advantage: Subword tokenization helps language models process unfamiliar words without requiring every word to be stored separately.

QUESTION 2: Token IDs and Vocabulary

A vocabulary contains tokens and their corresponding numerical IDs. The tokenizer converts text into these IDs before sending it to the Transformer.

Example:

Tokens: [“Machine”, “learning”, “works”]

Token IDs: [34, 17, 62]

These numbers are only examples; actual IDs depend on the model's vocabulary.

When a word is not available as a complete token, the tokenizer may divide it into smaller parts or use an unknown-token symbol.

Conclusion: Token IDs provide a numerical representation of text that a Transformer can process.

QUESTION 3: Word Embeddings

Word embeddings convert token IDs into vectors containing numerical values. These vectors are learned during model training and help the model identify useful relationships between tokens.

Process:

Token ID → Embedding Layer → Numerical Vector

For example, the tokens “doctor” and “hospital” may develop related representations because they often occur in similar contexts.

Token IDs themselves do not represent meaning. Instead, the learned embedding vectors provide information that the model can use. Positional information is also incorporated to represent the order of tokens.

Conclusion: Embeddings transform discrete token IDs into meaningful numerical representations for neural-network processing.

QUESTION 4: Transformer Encoder Architecture

A Transformer encoder processes input tokens and generates contextual representations. Each token can gather information from other tokens through self-attention.

Architecture:

Input Tokens
↓
Token Embeddings + Positional Information
↓
Multi-Head Self-Attention
↓
Residual Connection and Layer Normalization

↓

Feed-Forward Neural Network

↓

Residual Connection and Layer Normalization

↓

Contextual Representations

Functions of the components:

- **Self-attention:** Identifies relationships between tokens in the input.
- **Multi-head attention:** Examines different relationships using multiple attention heads.
- **Feed-forward network:** Applies learned transformations to each token representation.
- **Residual connection:** Adds the original input to a sublayer's output.
- **Layer normalization:** Helps maintain stable hidden representations during training.

Conclusion: The encoder produces context-aware representations that can support tasks such as classification and language understanding.

QUESTION 5: Query, Key, and Value Vectors

Query, Key, and Value vectors are the three main components used in the attention mechanism.

- **Query (Q):** Represents the information a token is searching for.
- **Key (K):** Helps determine how relevant a token is to another token.
- **Value (V):** Contains the information used to build the attention output.

The vectors are obtained by applying learned linear transformations to the input representations.

[
Q=XW_Q,\quad K=XW_K,\quad V=XW_V
]

The scaled dot-product attention formula is:

[
Attention(Q,K,V)=softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V
]

For example, in “The student opened the book because it was useful,” the model can use attention to connect relevant words and phrases. The exact attention pattern depends on the trained model.

Conclusion: Query, Key, and Value vectors allow the Transformer to determine which contextual information should contribute to each token's representation.

QUESTION 6: Attention Weights and Context

Attention weights indicate how strongly a token uses information from other token positions within a particular attention head.

Consider the sentence: “The teacher praised the student because she worked hard.”

For illustration, suppose the token “she” assigns the following hypothetical weights:

Token	**Example Attention Weight**
The	0.03
teacher	0.12
praised	0.05
the	0.02
student	0.50
because	0.08
she	0.05
worked	0.10
hard	0.05
Total	**1.00**

In this example, “student” receives the largest weight. This illustrates how attention may connect a pronoun with a relevant noun.

These values are imaginary and are not actual measurements from a trained model. Attention weights alone do not prove that the model has correctly understood a sentence.

Conclusion: Attention weights help illustrate how information from different tokens contributes to contextual representations.

QUESTION 7: Comparison of BERT, GPT, and T5

BERT, GPT, and T5 are Transformer-based models designed with different architectures and training objectives.

Feature	BERT	GPT	T5
Architecture	Encoder-only	Decoder-only	Encoder–decoder
Attention	Bidirectional self-attention	Causal self-attention	Encoder self-attention and decoder cross-attention
Main approach	Learns contextual representations	Predicts subsequent tokens	Converts tasks into text-to-text generation
Typical application	Sentiment classification	Story or paragraph generation	Text summarization

BERT: Uses information from both directions of the input to build contextual representations. It is commonly used for text classification.

GPT: Predicts the next token using the preceding context. It is widely used for generating text and answering questions.

T5: Encodes the input and generates an output sequence. It is suitable for summarization, translation, and other text-to-text tasks.

Conclusion: All three use Transformer technology, but their architectures and training methods make them suitable for different applications.

QUESTION 8: Transformer Text-Generation Pipeline

A Transformer language model generates text by processing an input prompt and repeatedly predicting subsequent tokens.

Steps involved:

1. **Input prompt:** Receives the initial text.
2. **Tokenization:** Divides the text into tokens.
3. **Token ID conversion:** Assigns a numerical ID to each token.
4. **Embedding generation:** Converts token IDs into vectors and incorporates positional information.
5. **Transformer processing:** Uses attention and feed-forward layers to process the sequence.
6. **Logit calculation:** Produces scores for possible next tokens.

7. **Token selection:** Uses a decoding strategy to select the next token.
8. **Repeated generation:** Adds the selected token to the sequence and continues until a stopping condition is reached.

Pipeline:

Input Text → Tokenization → Token IDs → Embeddings → Transformer Layers → Logits → Next Token → Repeat

Conclusion: Autoregressive text generation produces a response one token at a time until the model reaches an appropriate stopping point.

QUESTION 9: Transformer Block Diagram for Output Generation

A decoder-only Transformer generates text by predicting one token at a time from the available context.

Block Diagram:

Input Sentence
↓
Tokenizer
↓
Numerical Token IDs
↓
Embedding and Positional Information
↓
Masked Self-Attention
↓
Feed-Forward Network
↓
Output Projection
↓
Probability Distribution
↓
Next Token Selection
↓

Update Input Sequence

↓

Generate the Next Token Repeatedly

↓

Final Generated Text

Explanation:

- **Tokenizer:** Splits the input into tokens.
- **Token IDs:** Represent tokens as integers.
- **Embedding layer:** Converts IDs into vectors.
- **Masked self-attention:** Allows each position to use permitted preceding context without accessing future tokens.
- **Feed-forward network:** Transforms the contextual representations.
- **Output projection:** Produces scores for vocabulary tokens.
- **Token selection:** Chooses or samples a token based on the decoding method.
- **Repeated generation:** Continues until an end token or another stopping rule is reached.

Conclusion: The decoder-only Transformer uses masked attention and repeated next-token prediction to generate coherent text sequences.

QUESTION 10: Worked Example — "Artificial Intelligence supports learning."

Step 1: Tokenization

The input sentence is divided into tokens.

Example tokens: ["Artificial", "Intelligence", "supports", "learning", "."]

The actual tokenization may differ across models.

Step 2: Token ID Conversion

Each token is mapped to a numerical ID.

Illustrative token IDs: [120, 245, 318, 427, 105]

These numbers are hypothetical and do not represent verified IDs from a particular tokenizer.

Step 3: Embedding Generation

The token IDs are passed through the embedding layer to obtain vectors.

Token IDs → Embedding Vectors → Positional Information

The resulting representations contain information that the Transformer can process.

Step 4: Self-Attention

Self-attention allows each token to gather information from relevant tokens in the sentence.

For example, the representation of "supports" can incorporate information from "Artificial Intelligence" and "learning." This helps the model represent relationships between the words.

Step 5: Next-Token Prediction

The final contextual representation is transformed into scores for possible next tokens. A softmax operation can convert these scores into probabilities.

Suppose the following hypothetical probabilities are produced:

Candidate Token	**Example Probability**
effectively	0.45
efficiently	0.25
students	0.20
Other tokens	0.10
Total	**1.00**

If greedy decoding is used, the model selects "effectively" because it has the highest probability in this illustrative distribution. Other decoding strategies may select a different token.

Conclusion

This example explains how a Transformer converts text into token IDs, maps them to vectors, processes relationships through self-attention, and predicts subsequent tokens. The example IDs and probabilities are illustrative only, not actual model outputs.